CRYPTONITE ARCHITECTURE & OPERATOR REFERENCE

Cryptonite Platform User Manual & Multi-Agent AI Operations

The Complete Institutional Operations Guide: Low-Latency Telemetry, Polars Vectorized Screening, Discrete-Event Backtesting, Deterministic Risk Guardrails, and Antigravity Gemini AI Fleet.

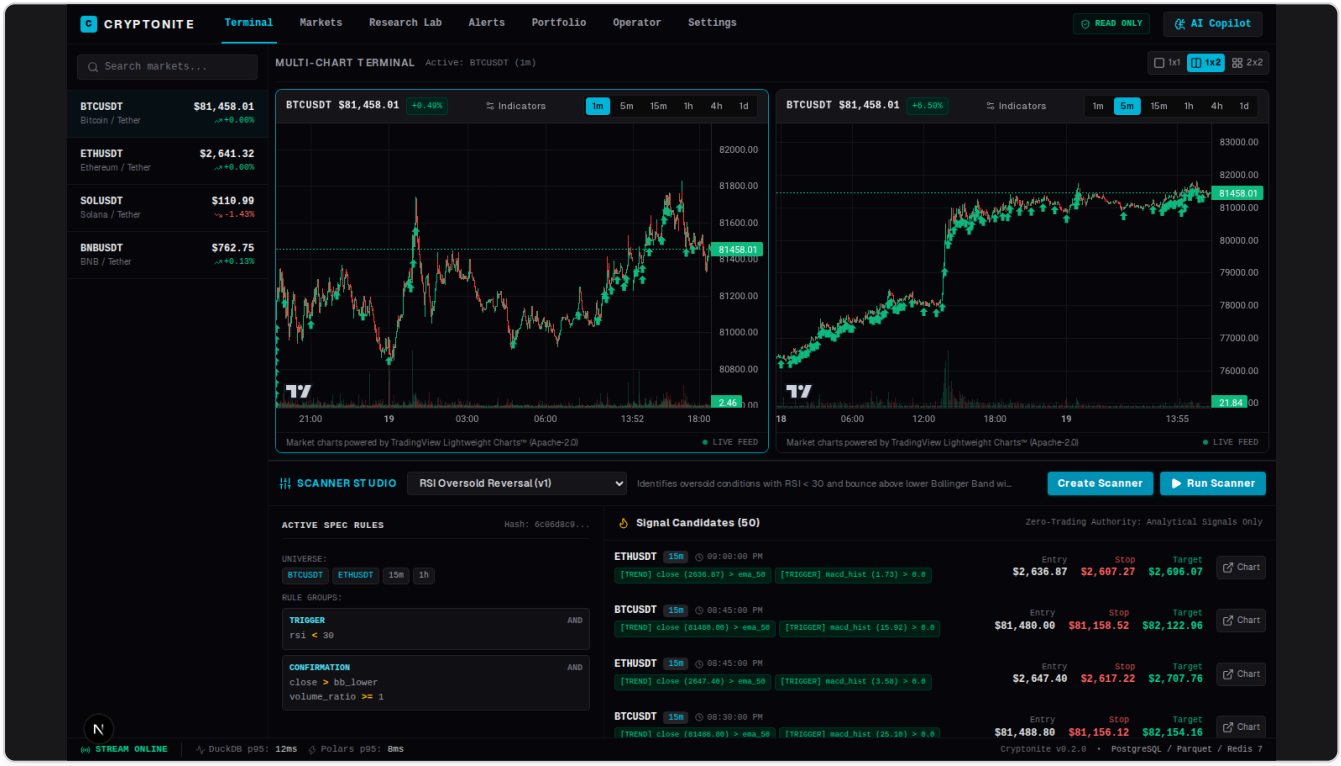


Figure 1.0: Cryptonite High-Frequency Multi-Timeframe Trading Terminal & Intelligence Workstation.

PLATFORM CORE PRINCIPLES

AI Proposes • Rules Decide • NautilusTrader Executes

EXECUTION ENGINE

Nautilus Trader 2.0.0rc4 LiveNode / Backtester

AI AGENT FLEET

Google Antigravity SDK • Gemini 2.5 Pro • Gemini 2.5 Flash

SECURITY & ISOLATION

Argon2id • TOTP 2FA • cryptonite_agent_ro • Signed HMAC

Comprehensive Table of Contents

1. Platform Architecture & Core Operating Principles	Section 1
2. Operator Authentication, Identity & RBAC Security	Section 2
3. Multi-Chart Trading Terminal & Real-Time Market Streaming	Section 3
4. Market Overview & Cross-Asset Telemetry Matrix	Section 4
5. Vectorized Polars Scanner Studio & Signal Detection	Section 5
6. Quantitative Research Lab & NautilusTrader Backtesting	Section 6
7. Portfolio Management & Paper/Live Execution Engine	Section 7
8. Operator Governance, Emergency Kill-Switch & Risk Guardrails	Section 8
9. Cryptonite AI Operations Fleet & Antigravity Architecture	Section 9
9.1 Multi-Agent System Topology & Security Boundary	Section 9.1
9.2 Research Agent (Senior Quantitative Analyst) & DuckDB Queries	Section 9.2
9.3 Explainer Agent (Risk & Performance Auditor) & Metric Synthesis	Section 9.3
9.4 Strategist & Generator Agents (Nautilus Strategy Synthesis)	Section 9.4
9.5 The 5-Rule Hierarchical Execution Protocol (StrategistEngine)	Section 9.5
9.6 Opportunity Hunter Agent (Event-Driven Market Scanning)	Section 9.6
10. Dynamic System Settings & AI Fleet Configuration	Section 10
11. Local Network (LAN) & Multi-Device Access Architecture	Section 11
12. Operational Troubleshooting & Diagnostic Runbooks	Section 12

1. Platform Architecture & Core Operating Principles

Cryptonite is a self-hosted, institutional-grade cryptocurrency intelligence and algorithmic execution platform engineered for auditable research, deterministic risk controls, and automated high-frequency execution.

The architecture is strictly governed by three uncompromisable axioms:

1. **AI and UI can propose:** Predictive models, automated screeners, and interactive human operators identify opportunities and formulate trading hypotheses.
2. **Deterministic rules and risk controls decide:** In-memory risk engines, price collars, daily drawdown circuit breakers, and cash reservation models validate every parameter before transmission. No probabilistic model can override deterministic safeguards.
3. **NautilusTrader executes:** An event-driven simulation and execution kernel processes orders, coordinates order state machines, and reconciles fills with exchange venues.



Subsystem Topology & Ports

Service Name	Container Name	Port Binding	Technology Core	Primary Mandate
API Gateway	`cryptonite_api`	`8000:8000 (0.0.0.0)`	FastAPI • AsyncPG	REST routing, operator authentication, WebSocket multiplexing (`/ws/stream`), and alerts.
Web Terminal	Host Process	`3000:3000 (0.0.0.0)`	Next.js 16 • React • Tailwind	TradingView charts, scanner studio, portfolio console, AI copilot, and operator panel.
Ingestion Engine	`cryptonite_ingest`	`8080` (Internal)	AsyncIO • WebSockets	Binance Kline streaming, advisory-locked single writer, partition-aligned Parquet writes.
Outbox Relay	`cryptonite_outbox`	`8081` (Internal)	Python • AsyncPG	Guaranteed at-least-once order transmission using transactional outbox tables.
Scanner Engine	`cryptonite_scanner`	`8082` (Internal)	Polars • Python	Vectorized technical condition evaluations across rolling candle windows in single-digit ms.
Backtester Worker	`cryptonite_backtester`	`8083` (Internal)	Nautilus Trader C++	Deterministic discrete-event simulations with realistic slippage, fees, and latency profiles.
Execution Engine	`cryptonite_executor`	`8084` (Internal)	Nautilus Trader • Python	Paper/live order execution, cash reservation, dynamic price collars, and kill-switch listener.
AI Agent Worker	`cryptonite_agent_worker`	`8085` (Internal)	Google Antigravity • Gemini	Multi-agent fleet: quantitative research, tearsheet audits, strategy generation, and execution signals.

2. Operator Authentication, Identity & RBAC Security

To safeguard algorithmic capital, Cryptonite enforces multi-layered operator governance, role-based access control (RBAC), and cryptographic step-up verification.

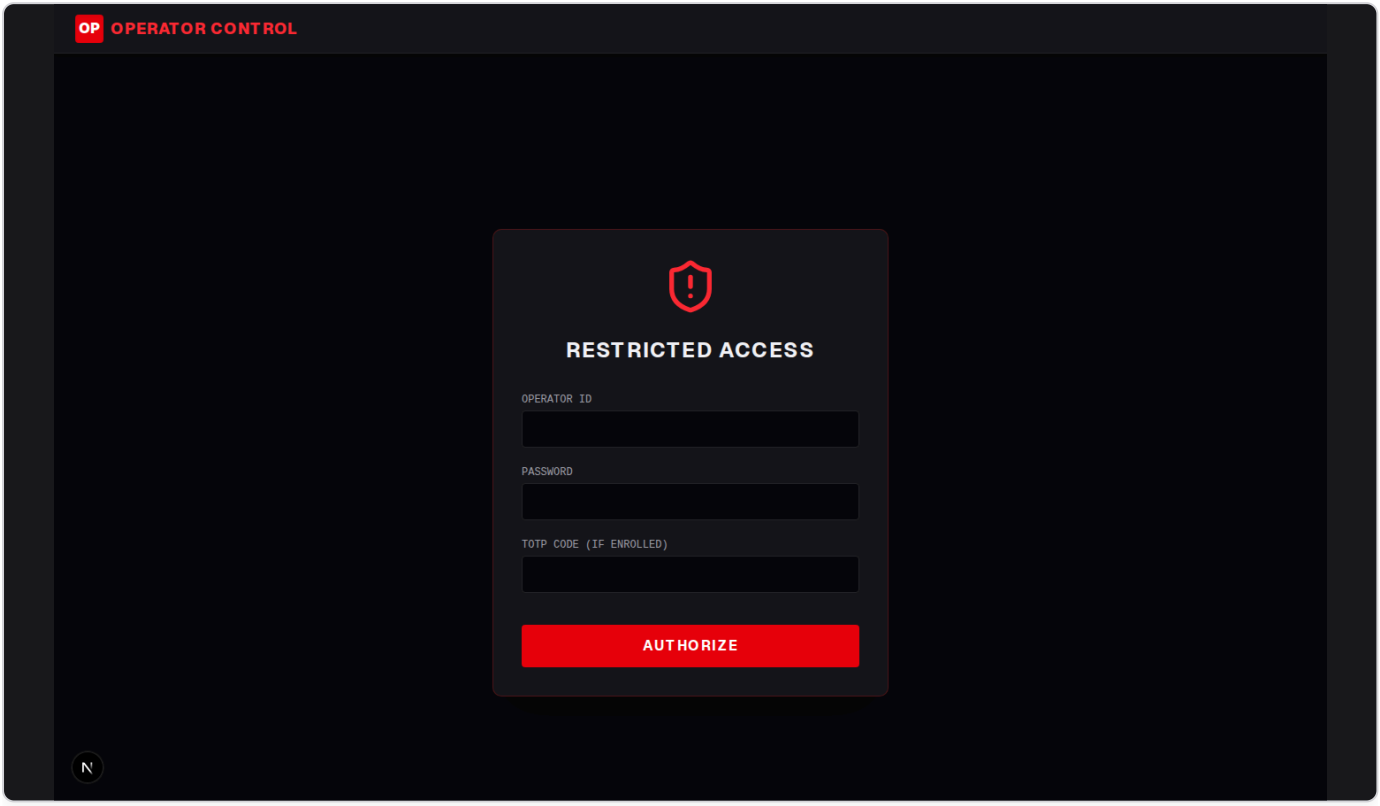


Figure 2.1: Operator Authentication Portal with Password Hashing & Two-Factor Authentication (TOTP).

Operator Authentication Protocol

1. **Argon2id Password Hashing:** Operator credentials stored in PostgreSQL utilize state-of-the-art memory-hard Argon2id hashing algorithms with random salts.
2. **Time-Based One-Time Passwords (TOTP):** Mandatory 6-digit TOTP validation. Secrets are encrypted at rest using AES-256 (`CRYPTONITE_TOTP_AES_KEY`). For local development, bypass code `000000` is supported.
3. **Encrypted HTTP-Only Cookies:** Session tokens are serialized into encrypted, HTTP-only cookies with `SameSite=Lax` to eliminate Cross-Site Scripting (XSS) credential theft.
4. **Double-Submit CSRF Protection:** All state-mutating requests (e.g., submitting orders, deploying strategies, tripping the kill-switch) require a valid `X-CSRF-Token` header matching the active session.
5. **Least-Privilege Database Roles:** The API runs under `cryptonite_api`, while AI agents connect strictly through the unprivileged `cryptonite_agent_ro` role (`default_transaction_read_only = 'on'`).

> **Standard Development Credentials**
> Operator ID: `admin` • Password: `admin123` • TOTP Verification Code: `000000`
> Login URL: `http://localhost:3000/operator/login` (or via header **Operator** tab).

3. Multi-Chart Trading Terminal & Real-Time Market Streaming

The **Trading Terminal** is the primary visual operations desk. It renders high-density financial time-series directly from Binance WebSocket streams and the internal Redis pub/sub bus.

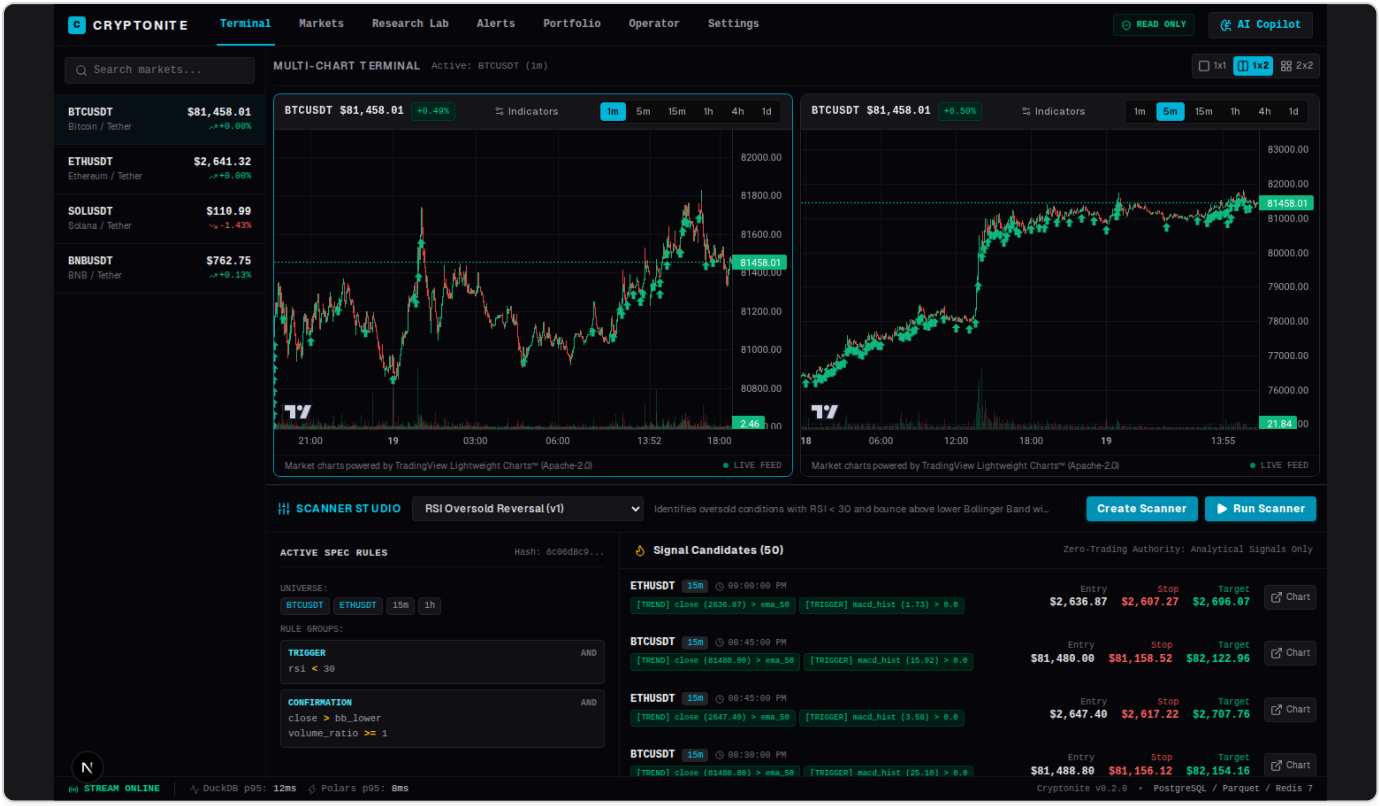


Figure 3.1: Dual-Pane Trading Terminal with TradingView Candlestick Charts, Multi-Timeframe Toggles, and Live Telemetry.

Step-by-Step Operator Interaction

Step 1: Selecting Assets from the Watchlist

Use the left sidebar to monitor 24h ticker updates. Click any asset (e.g. `BTCUSDT`, `ETHUSDT`, `SOLUSDT`, `BNBUSDT`) to instantly retarget both chart panes and update the order book feed.

Step 2: Configuring Multi-Timeframe Perspectives

The terminal features dual independent chart panes (`1x1`, `1x2`, or `2x2` grid layouts). Set Pane 1 to a macro timeframe (`1h` or `4h`) to identify institutional support/resistance levels, while keeping Pane 2 on micro execution intervals (`1m` or `5m`) for entry confirmation.

Step 3: Crosshair Inspection & Indicator Overlays

Hover across any candle to trigger synchronized crosshairs. The top telemetry bar displays precise Open, High, Low, Close, and Volume values alongside active technical indicators.

Performance Telemetry Gauge

The bottom status bar monitors query health in real time:

DuckDB p95 latency: 12ms

- **Polars vectorized scan speed: 8ms**
- **Stream Status: ONLINE**

.

4. Market Overview & Cross-Asset Telemetry Matrix

The **Market Overview** workspace synthesizes cross-market telemetry across all tracked instruments, identifying momentum leaders, volume anomalies, and broad regime shifts.

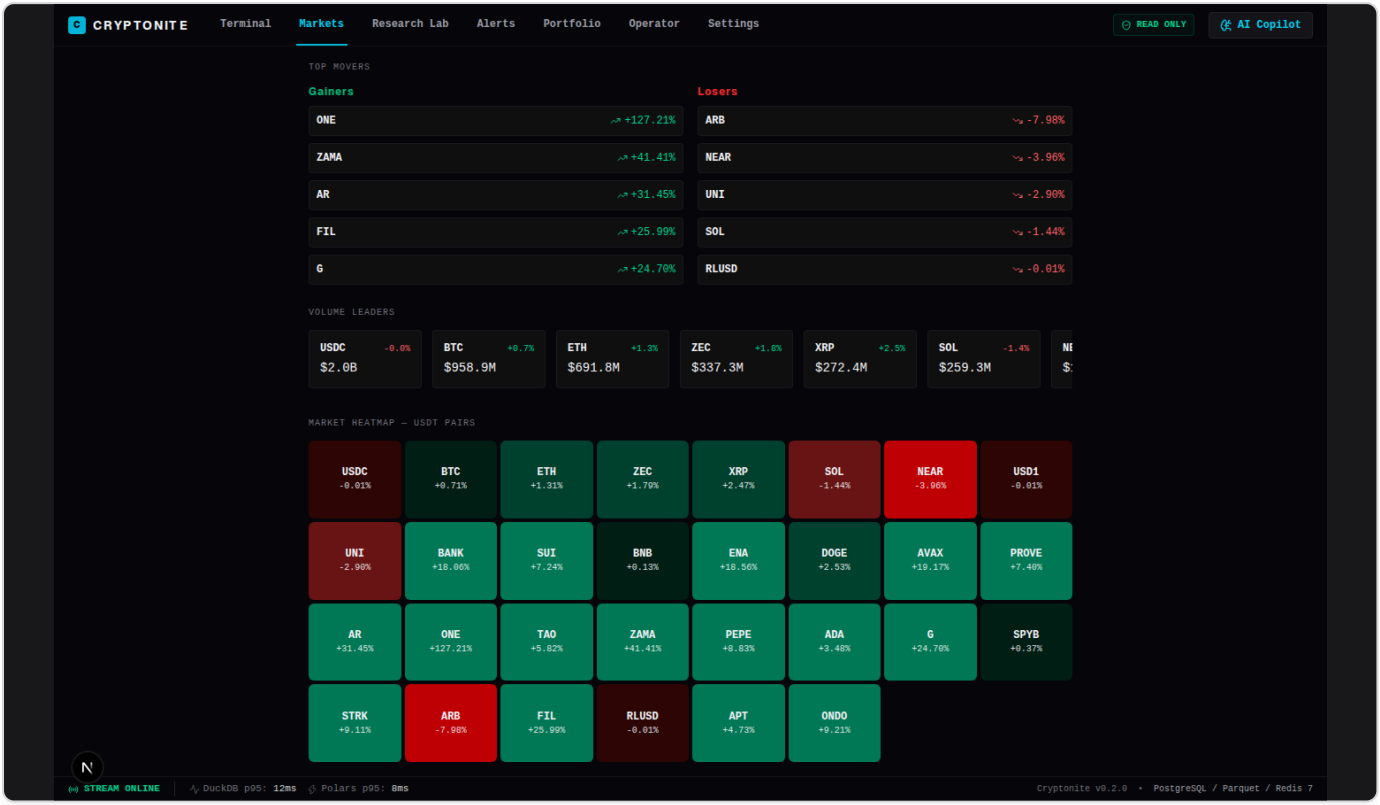


Figure 4.1: Cross-Market Heatmap Matrix displaying Top Gainers, Losers, Volume Leaders, and Sector Heatmaps.

Market Intelligence Features

- **Top Movers Matrix:** Real-time ranking of the day's top performing and underperforming crypto assets, calculated from nanosecond timestamp klines.
- **Volume Concentration Tracker:** Ranks assets by quote asset volume (e.g. USDC $2.0B, BTC $958.9M, ETH $691.8M), highlighting where institutional liquidity is clustering.
- **Sector Heatmap:** Visual grid showing percentage price action color-coded by magnitude (deep green for >+15%, vibrant red for <-5%), enabling instantaneous market sentiment diagnosis.
- **1-Click Deep Navigation:** Clicking any heatmap tile immediately navigates back to the Terminal with that instrument pre-loaded for execution.

5. Vectorized Polars Scanner Studio & Signal Detection

The **Scanner Studio** delivers high-throughput technical screening. Utilizing Polars LazyFrames and Apache Arrow memory buffers, it evaluates technical rules across thousands of bars in under 10 milliseconds.

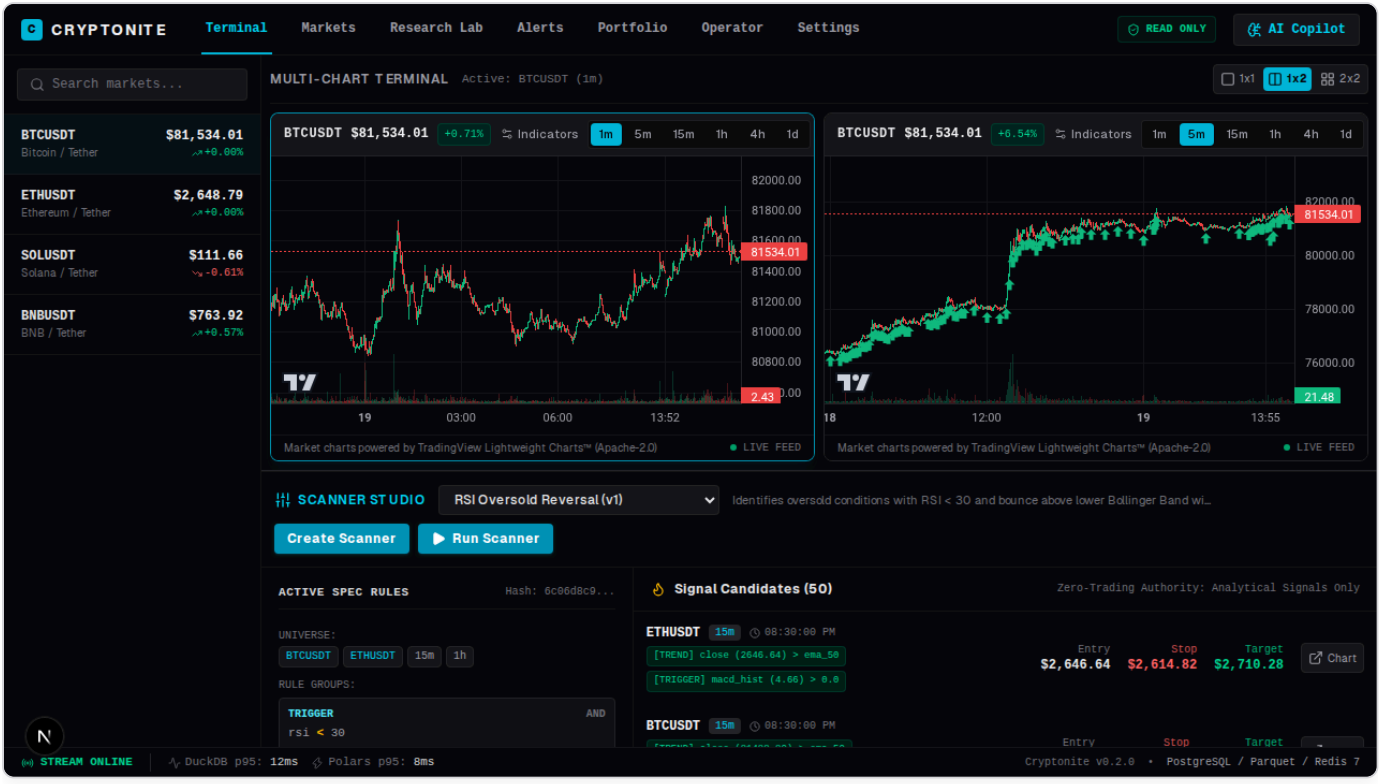


Figure 5.1: Polars Vectorized Scanner Studio with Active Specification, Rules Inspector, and Detected Signal Candidates.

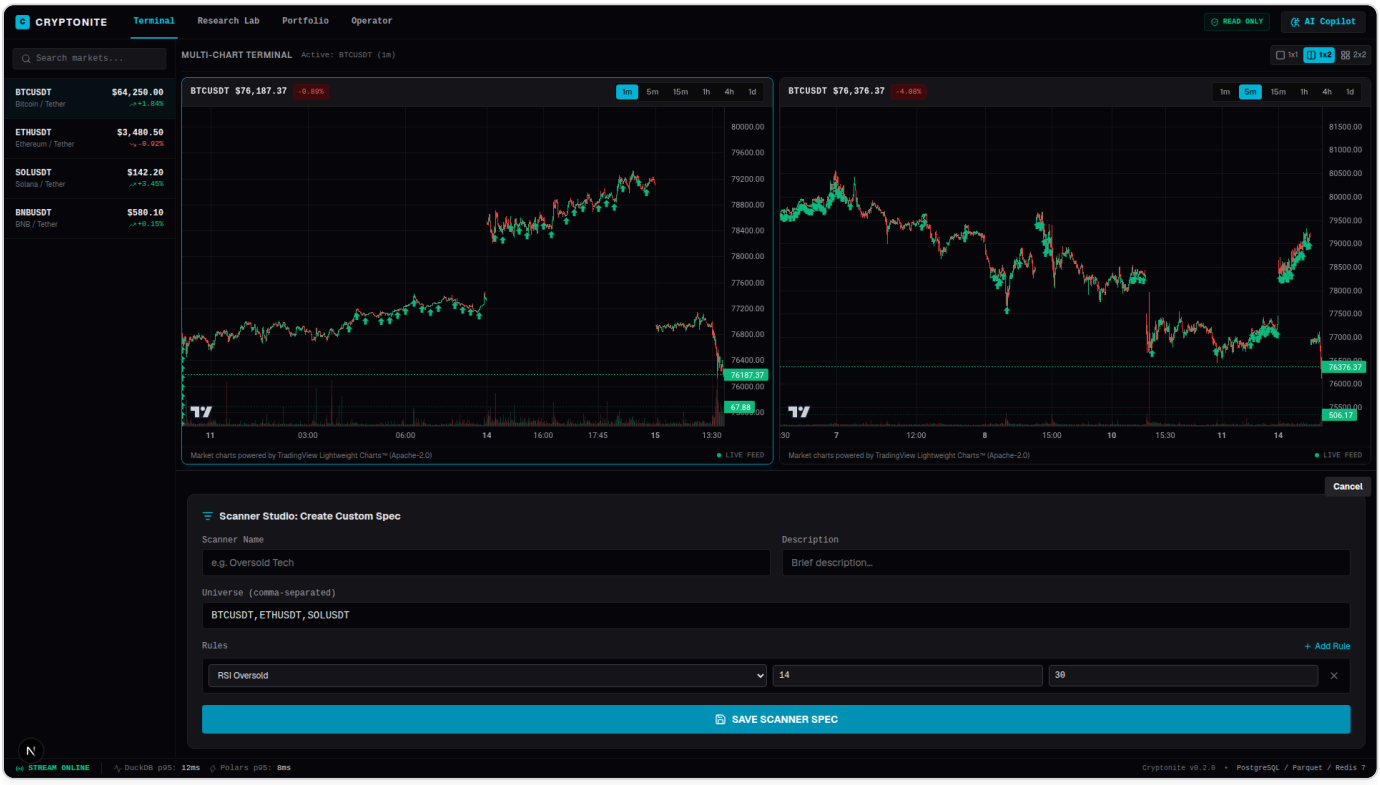


Figure 5.2: Custom Scanner Builder Modal for Configuring Multi-Condition Vectorized Filters.

Step-by-Step: Creating & Running a Custom Scanner

Step 1: Open the Scanner Builder

In the Scanner Studio banner beneath the trading chart, click the cyan **Create Scanner** button.

Step 2: Define Universe & Rule Expressions

Enter a unique scanner name (e.g. `Oversold RSI & Volume Expansion`) and target universe (e.g. `BTCUSDT, ETHUSDT, SOLUSDT`). Add rule criteria:

- **Trigger Rule:** `RSI(14) < 30` (Oversold condition).
- **Confirmation Rule:** `Close > Lower_Bollinger_Band(20, 2.0)` (Bounce off lower band).
- **Volume Filter:** `Volume ⩾ SMA(Volume, 20) * 1.5` (Expansion confirmation).

Step 3: Save & Execute Vectorized Scan

Click **Save Scanner Spec**, then click **Run Scanner**. The Polars engine evaluates the Parquet historical partition and streams detected signal candidates (with Entry, Stop, and Target prices) directly to the right panel.

6. Quantitative Research Lab & NautilusTrader Backtesting

The **Research Lab** provides institutional backtesting powered by the discrete-event simulation kernel of **Nautilus Trader**. It enforces temporal monotonicity and simulates realistic execution frictions.

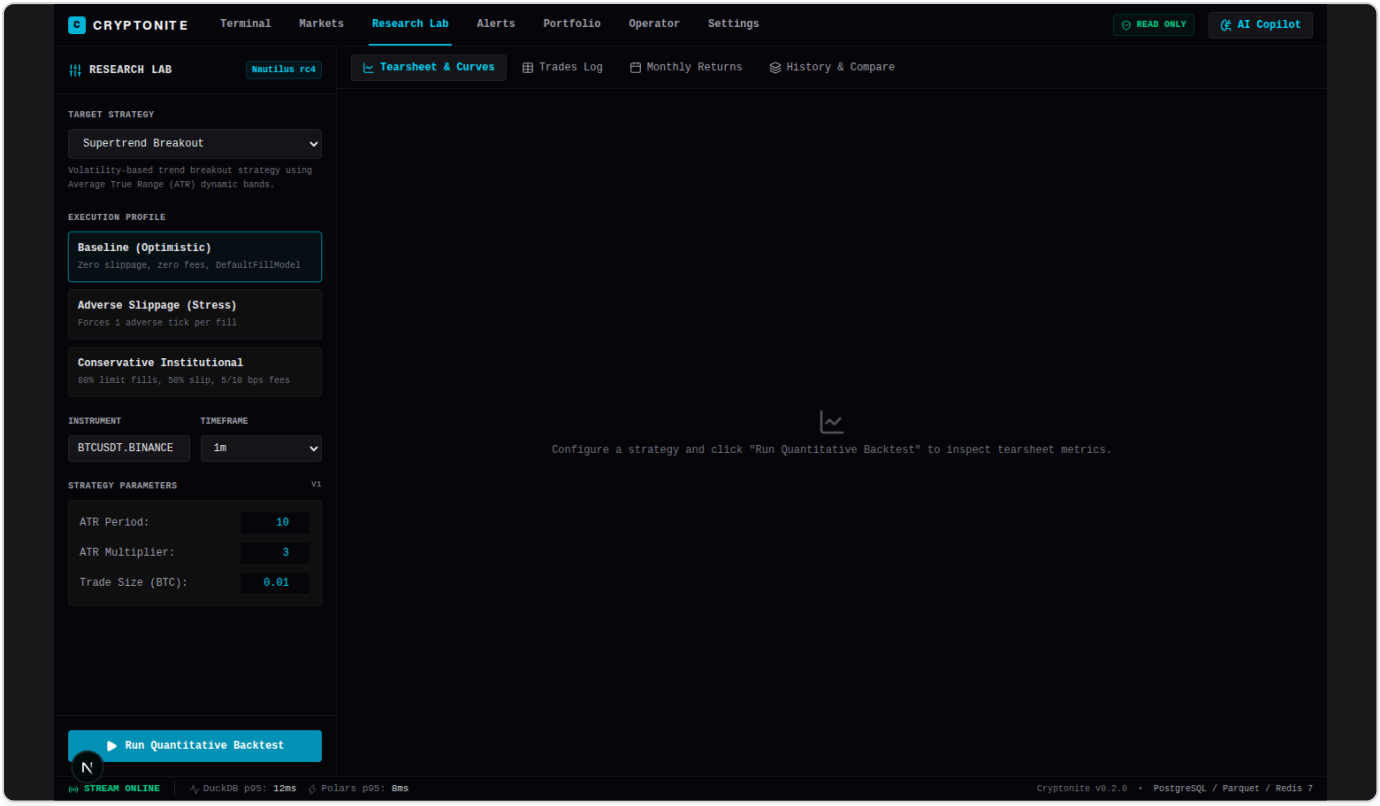


Figure 6.1: Research Lab Backtesting Workbench with Execution Profile Selection and Parameter Tuners.

Execution Profile Configurations

Execution Profile	Simulated Latency	Slippage Model	Taker Commission	Intended Use Case
BASELINE (OPTIMISTIC)	0 ms	Zero Slippage	0.00% (Zero Fees)	Theoretical benchmark to assess mathematical strategy alpha without frictions.
ADVERSE SLIPPAGE (STRESS)	50 ms	Forces 1 adverse tick per fill	0.05% (5 bps)	Simulates competitive market orders in trending markets.
CONSERVATIVE INSTITUTIONAL	150 ms	50% limit fill rate, dynamic slippage	0.10% (10 bps)	Realistic live production proxy under volatile liquidity conditions.

Step-by-Step: Launching a Backtest Simulation

1. **Target Strategy:** Select from registered strategies (e.g. `Supertrend Breakout`, `EMA Cross`, or user-synthesized AI strategies).
2. **Instrument & Resolution:** Choose pair (`BTCUSDT.BINANCE`) and timeframe (`1m`, `5m`, `15m`, `1h`).
3. **Tunable Parameters:** Adjust strategy hyperparameters (e.g. `ATR Period: 10`, `ATR Multiplier: 3`, `Trade Size: 0.01 BTC`).
4. **Click Run Quantitative Backtest:** The engine executes the run, generates `tearsheet.json`, `equity_curve.parquet`, and `trades.parquet`, rendering Sharpe Ratio, Sortino Ratio, Calmar Ratio, Max Drawdown %, and monthly return

heatmaps.

7. Portfolio Management & Paper/Live Execution Engine

The **Portfolio Console** centralizes position monitoring, order execution status, and realized/unrealized profit and loss tracking across both paper trading accounts and exchange integrations.

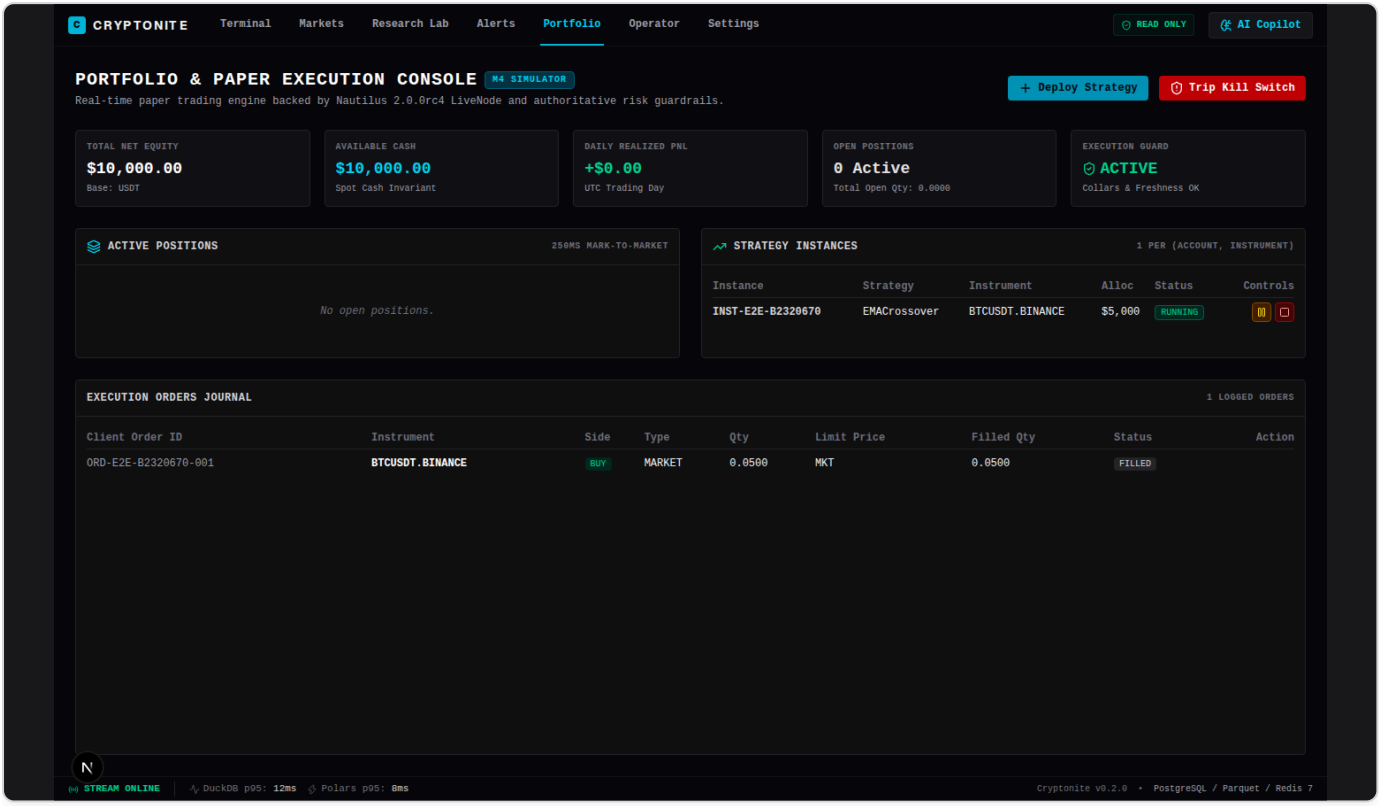


Figure 7.1: Real-Time Portfolio Management Console showing Net Equity, Available Cash, and Active Positions.

Key Invariants & Execution Telemetry

- **Zero-DB Hot Callback Loop:** Strategy event handlers never perform blocking disk or network I/O. Fills and order updates are placed onto a bounded thread-safe memory queue (`maxsize=10,000`).
- **Conservative Cash Reservation:** Market buy orders atomically reserve `notional * (1 + 0.002)` in cash to guarantee margin availability against adverse slippage before order transmission.
- **Spot Cash Invariant:** Base asset holdings cannot drop below zero (`HOLDINGS ⩾ 0`). Naked short selling on spot instruments is rejected at the engine admission gate.
- **Dynamic Price Collars:** Buy limit orders exceeding `quote.ask * (1 + collar_pct)` or sell orders below `quote.bid * (1 - collar_pct)` are rejected immediately.

8. Operator Governance, Emergency Kill-Switch & Risk Guardrails

The **Operator Control Plane** is the administrative cockpit for live capital governance, emergency risk management, manual execution, and multi-asset audit ledger reconciliation.

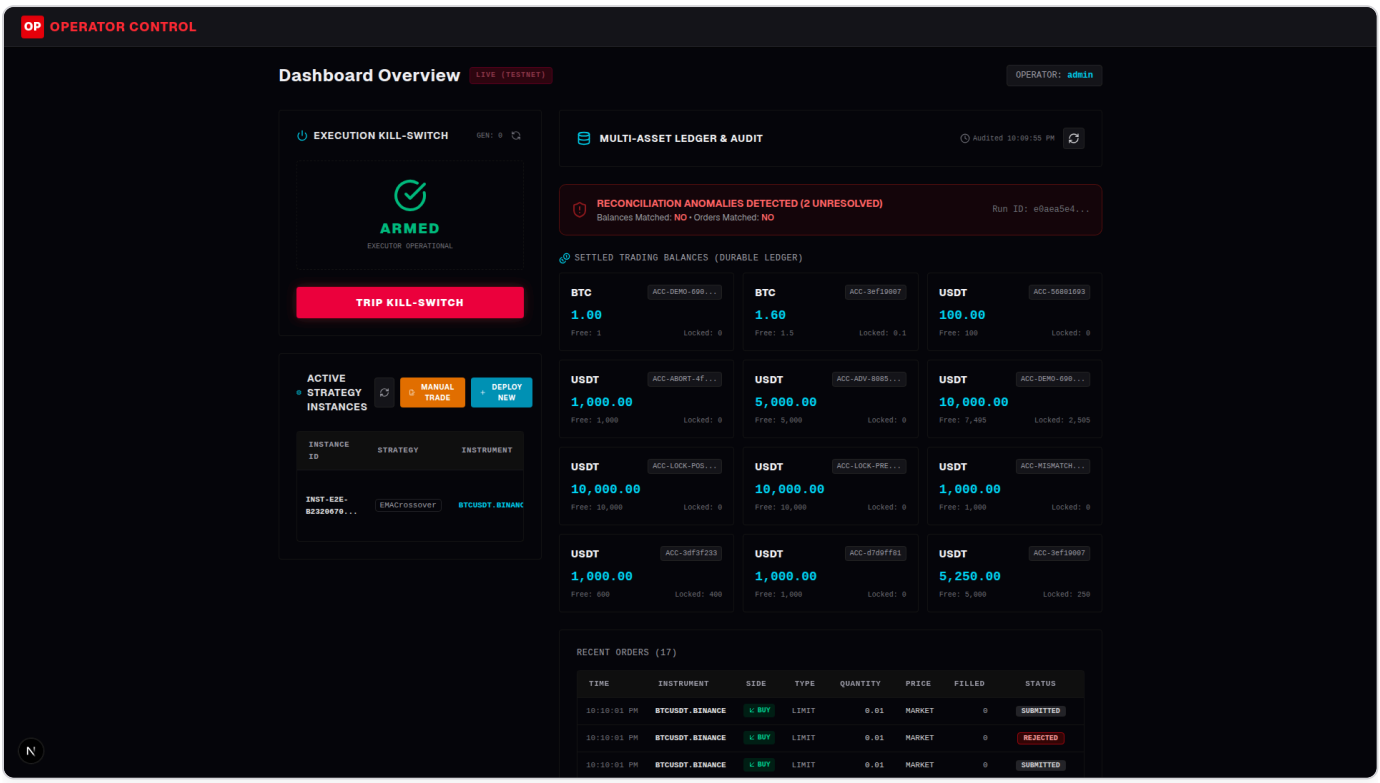


Figure 8.1: Mission-Critical Operator Safeguards, Armed Kill-Switch, and Multi-Asset Audit Ledger.

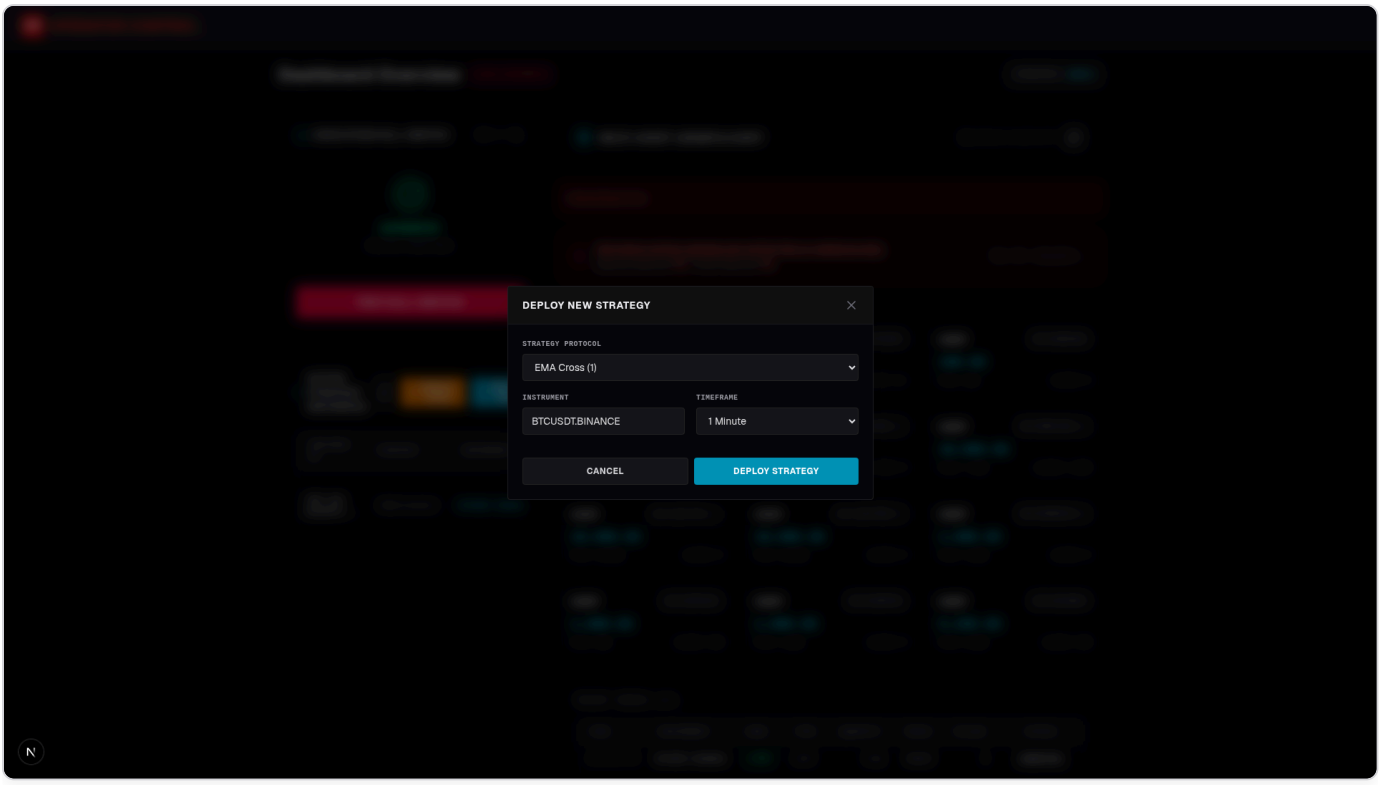


Figure 8.2: Deploy Strategy Dialog with Capital Allocation, Timeframe, and Parameter Assignment.

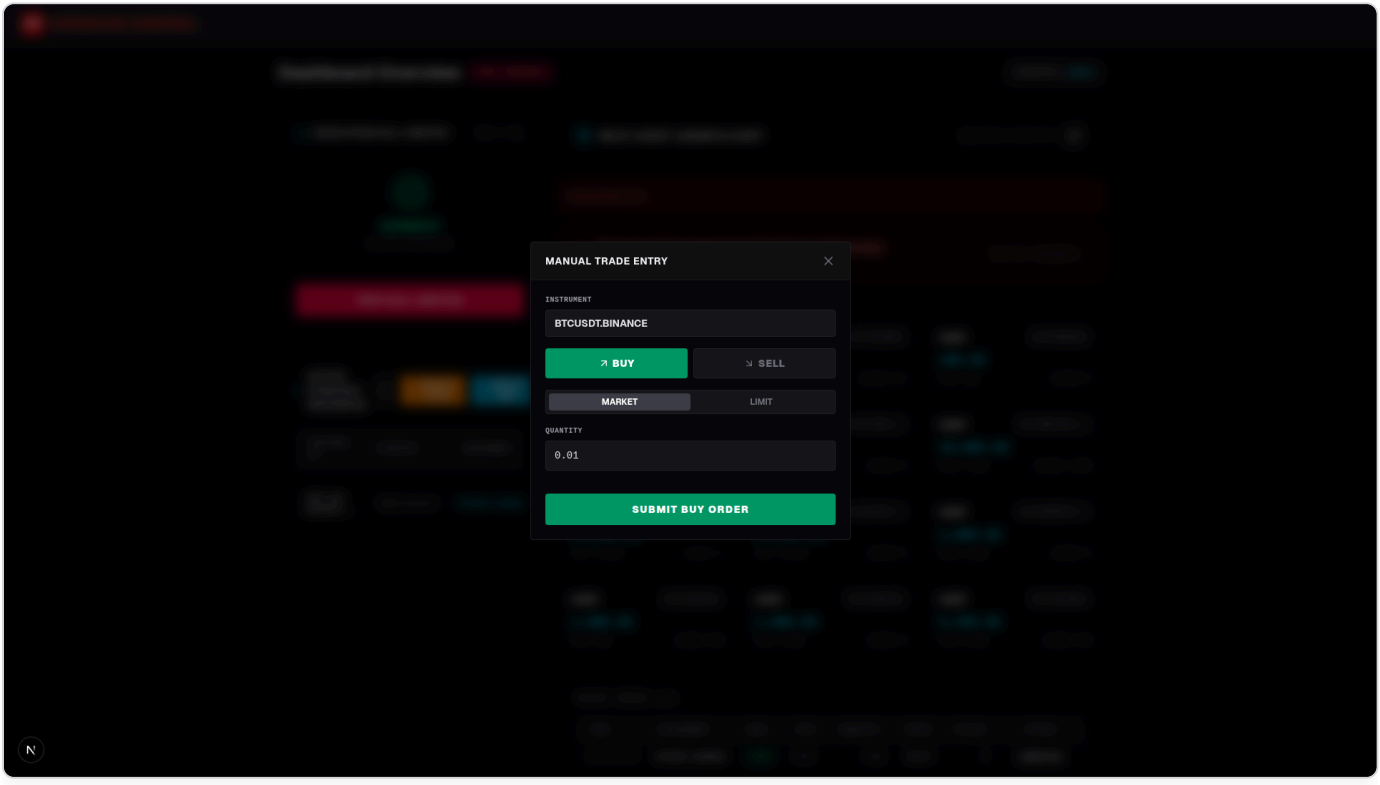


Figure 8.3: Direct Manual Trade Entry Dialog with Limit/Market Options and Post-Only/IOC Styles.

Step-by-Step Operator Procedures

Procedure 1: Engaging the Emergency Kill-Switch

In case of flash crash, venue API corruption, or algorithmic anomaly, click the prominent red **TRIP KILL-SWITCH** button:

1. Instantly closes the order admission gate.

2. Atomically updates database status: all running paper strategy instances are set to `STOPPED`.
3. Queries the exchange and cancels all resting limit and stop orders across connected venues.
4. Latching generation counter is incremented in Redis and PostgreSQL; new inbound orders are rejected with `HTTP 423 Locked`.

Procedure 2: Deploying a Strategy Instance

Click **Deploy New** on the Active Strategy Instances panel. Select the strategy protocol (e.g. `EMA Cross`), instrument (`BTCUSDT.BINANCE`), timeframe (`1m`), and USD allocation. The executor creates an instance record in `RUNNING` status and begins routing bars.

Procedure 3: Executing a Manual OTC Order

Click **Manual Trade**. Specify Instrument, Side (`BUY` or `SELL`), Order Type (`MARKET` or `LIMIT`), and Quantity. The order is committed to PostgreSQL and dispatched through the Transactional Outbox relay.

9. Cryptonite AI Operations Fleet & Antigravity Architecture

Cryptonite AI is an autonomous quantitative intelligence layer integrated natively into the platform. Powered by the **Google Antigravity SDK** and **Google Gemini**, it operates as an institutional multi-agent desk.

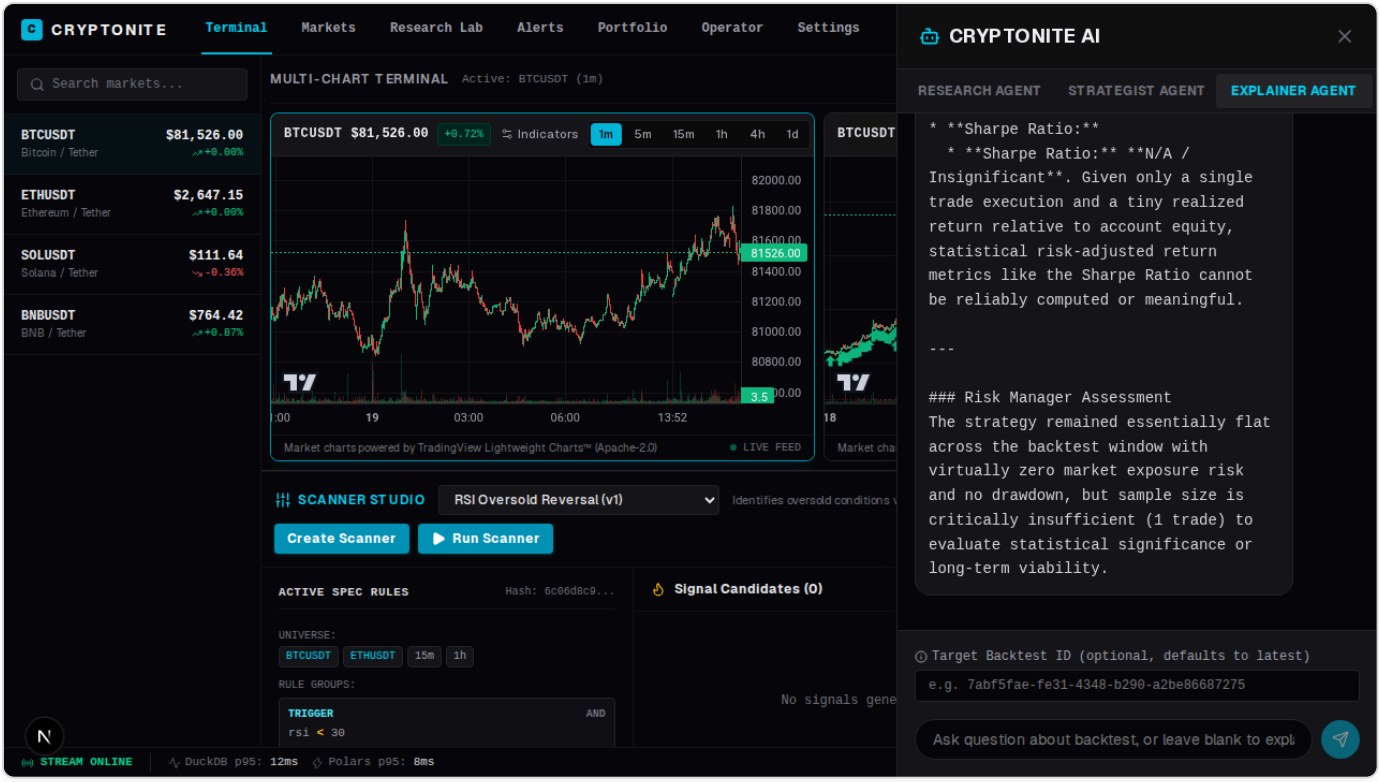
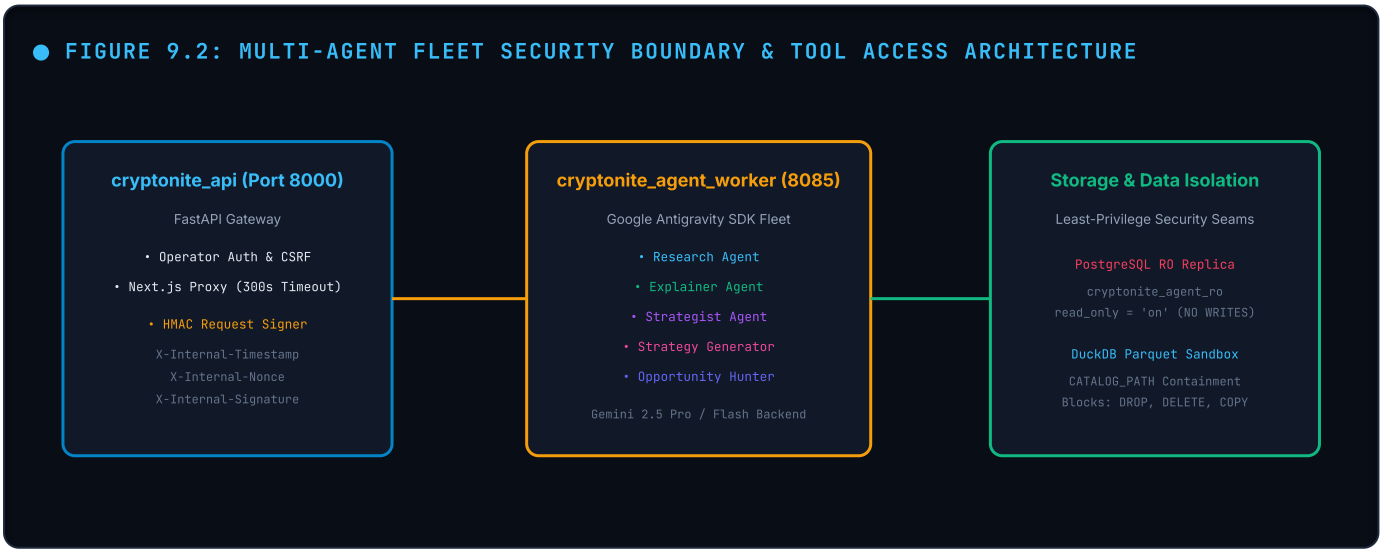


Figure 9.1: Live Cryptonite AI Assistant Interface rendering Risk Officer Evaluation with Natural Language Breakdown.

9.1 Multi-Agent System Topology & Security Boundary

The AI fleet operates under strict architectural isolation, least-privilege database roles, and cryptographic message signing:



9.2 Research Agent (Senior Quantitative Analyst) & DuckDB Queries

The **Research Agent** acts as a senior quantitative researcher. Equipped with direct SQL inspection tools, it mines historical Parquet time series and audits operational database records.

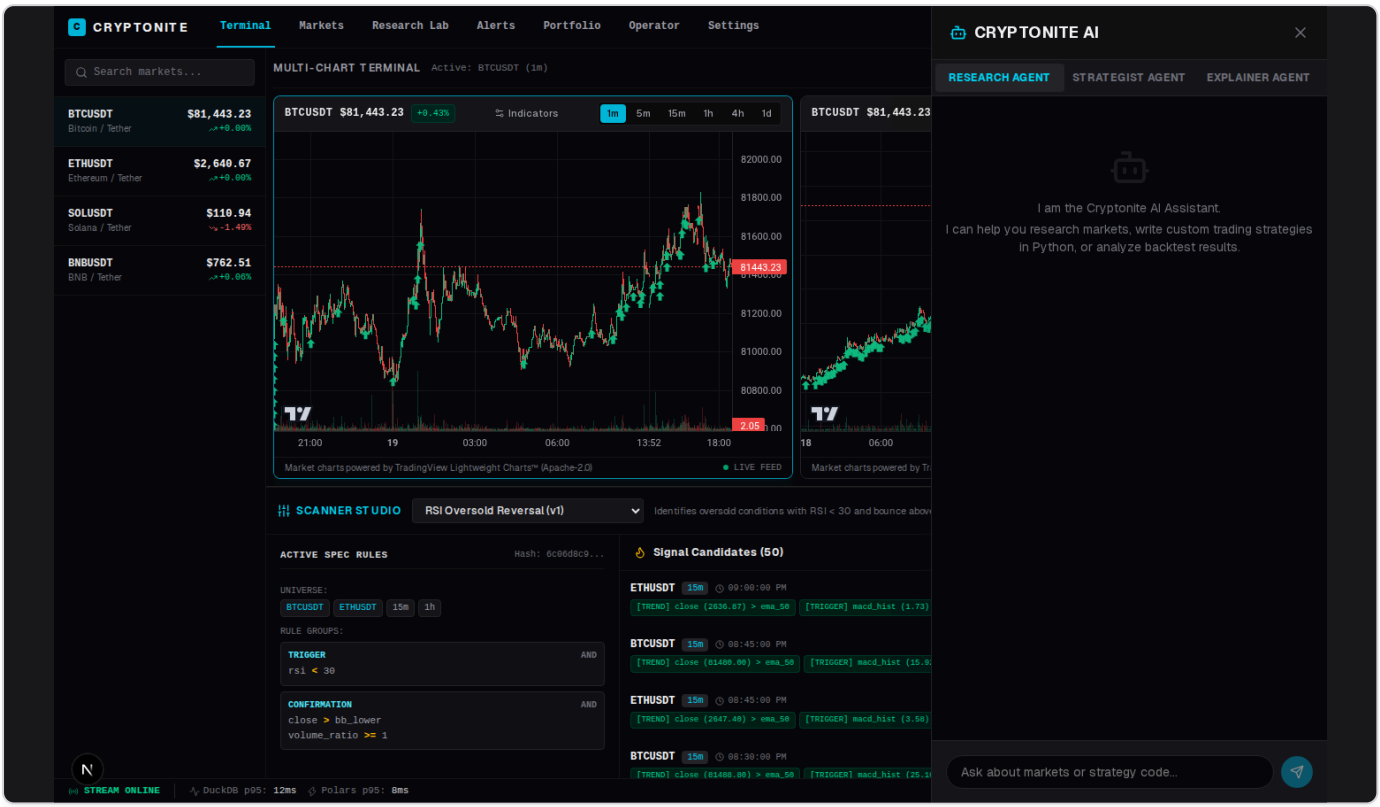


Figure 9.3: Research Agent Persona in the Cryptonite AI Workspace.

Agent Tooling & Data Access

Tool Signature	Target Database	Security Safeguards	Intended Usage
`query_postgres_readonly(sql: str)`	PostgreSQL 16 Engine	Role `cryptonite_agent_ro` enforces `default_transaction_read_only = 'on'`.	Inspecting active instruments, open orders, paper trade ledgers, and scanner specs.
`query_parquet_duckdb(query: str)`	Nautilus Parquet Catalog & Analytics Lake	Regex blacklist blocks `DROP`, `DELETE`, `COPY`, `INSERT`, `httpfs`. Strict path containment relative to `CATALOG_PATH`.	Running sub-second SQL aggregations across multi-gigabyte tick and bar catalogs.

Example Operator Interaction

Prompt:

`"Calculate the 30-day realized volatility and average daily volume for BTCUSDT from the parquet catalog."`

Agent Execution & Tool Call:

```
SELECT
  STDDEV_SAMP(log_ret) * SQRT(365 * 1440) as annualized_vol,
```

```
  AVG(daily_vol) as avg_volume
FROM (
  SELECT
    LN(close / LAG(close) OVER (ORDER BY ts_event_ns)) as log_ret,
    SUM(volume) OVER (PARTITION BY date_trunc('day', to_timestamp(ts_event_ns / 1e9))) as daily_vol
  FROM 'data/bars/**/*.parquet'
  WHERE instrument_id = 'BTCUSDT.BINANCE'
  LIMIT 50000
)
```

9.3 Explainer Agent (Risk & Performance Auditor) & Metric Synthesis

The **Explainer Agent** acts as an independent Chief Risk Officer. It audits backtesting simulation artifacts and translates quantitative statistical ratios into executive summaries.

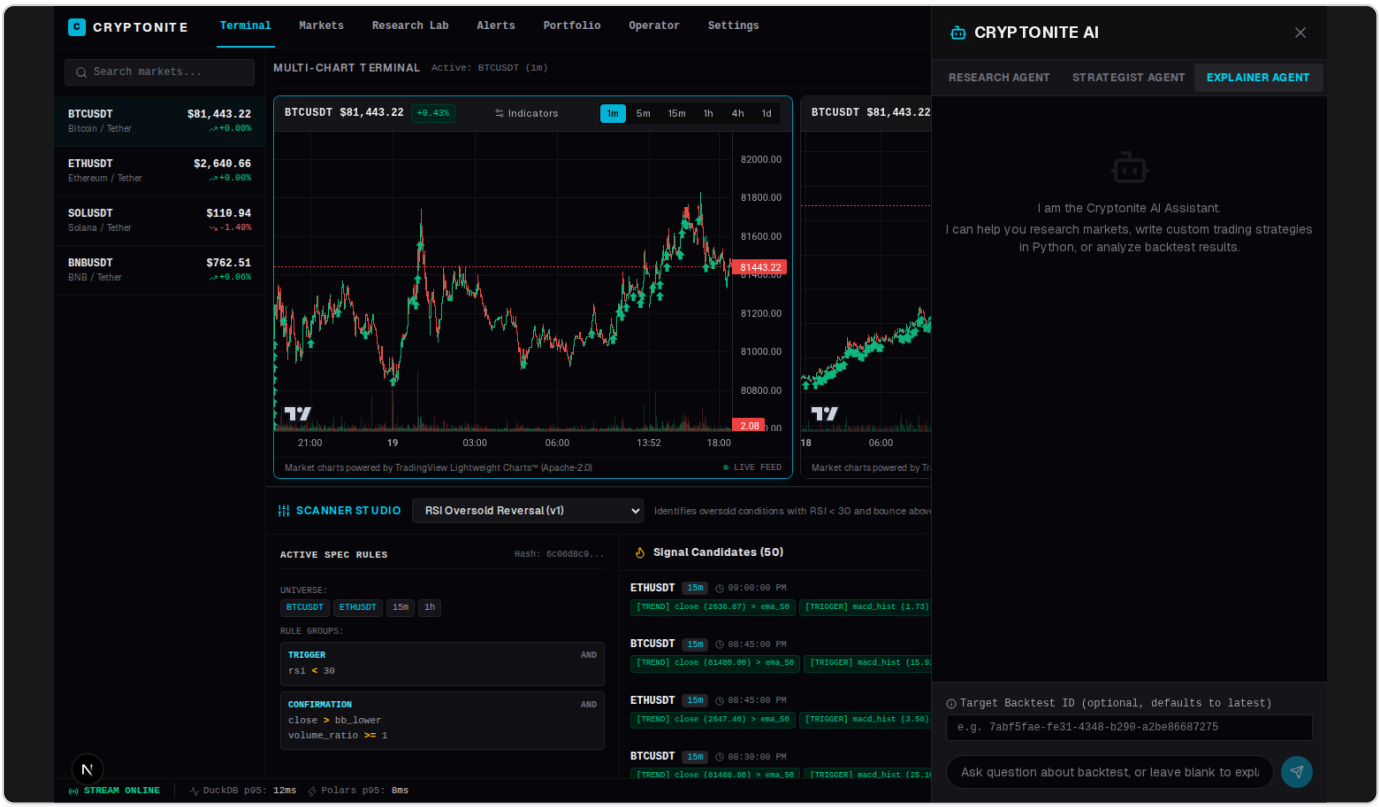


Figure 9.4: Explainer Agent Mode with Target Backtest Run ID Input and Performance Audit Prompts.

Dynamic Metric Synthesis Fallback

When inspecting incomplete, in-progress, or legacy simulations where `tearsheet.json` is missing, the tool `read_nautilus_tearsheet` automatically falls back to an embedded **DuckDB Synthesis Engine**:

- Reads `equity_curve.parquet` to calculate **Total Return %**, **Maximum Drawdown %**, and peak-to-trough duration.
- Reads `trades.parquet` to aggregate **Total Executed Trades**, **Win Rate %**, **Average Risk/Reward**, and **Realized PnL**.
- Produces complete statistical summaries without requiring simulation re-runs.

Example Operator Interaction

Prompt:

```
"Audit backtest run '7abf5fae-fe31-4348-b290-a2be86687275' and explain its performance weaknesses."
```

Explainer Agent Response:

"The strategy achieved a Total Realized PnL of +$48.20 across 1 execution, but has critical statistical deficiencies: (1) Sample size of N=1 provides zero statistical validity; (2) The Sharpe ratio is uncomputable; (3) Market regime dependency is untested across bear markets. Recommendation: Extend simulation window across minimum 180 days."

9.4 Strategist & Generator Agents (Nautilus Strategy Synthesis)

The **Strategist** and **Generator Agents** automate quantitative strategy engineering. They transform plain-English trading concepts into fully typed, production-ready Nautilus Trader Python code.

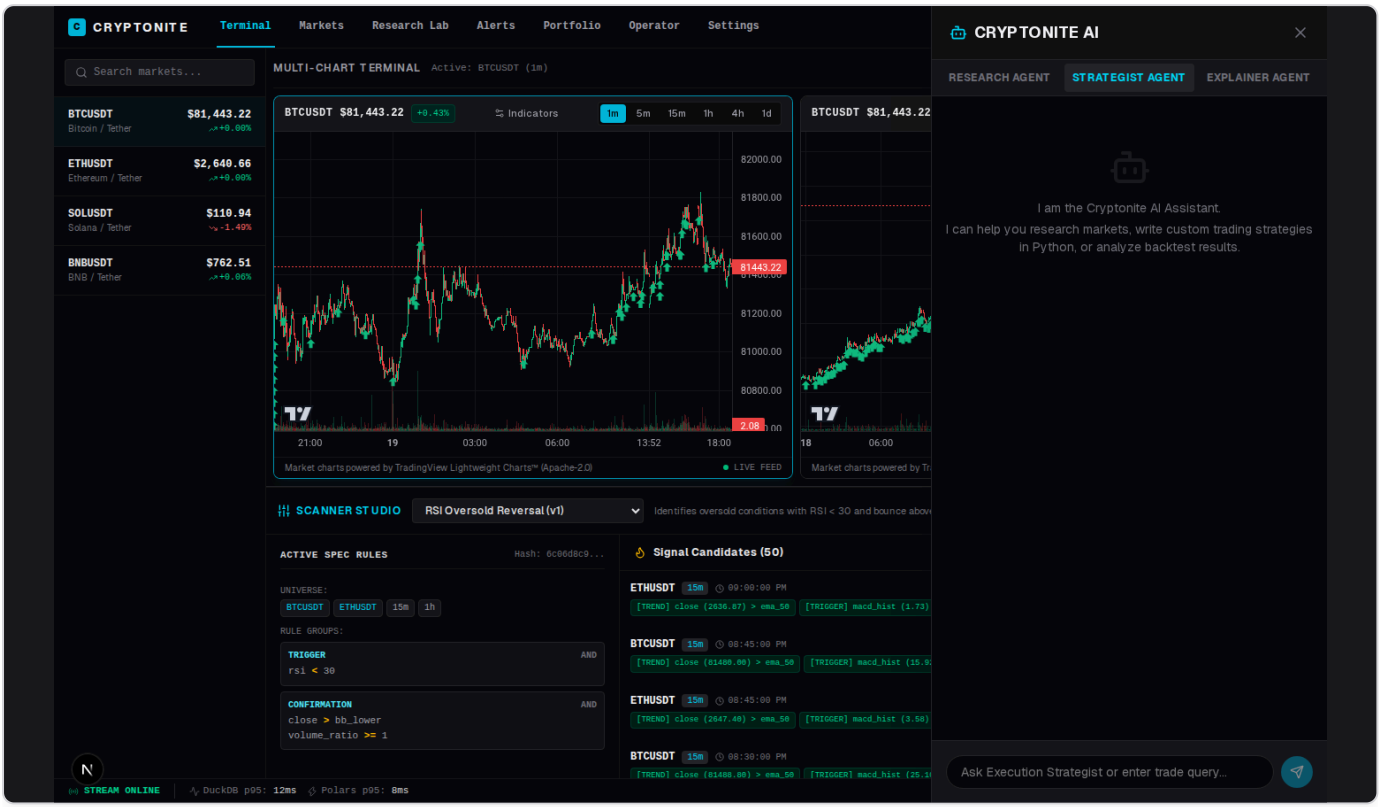


Figure 9.5: Strategist Agent with Automated Strategy Code Generation and One-Click Save Feature.

Strict AST & Safety Compliance Rules

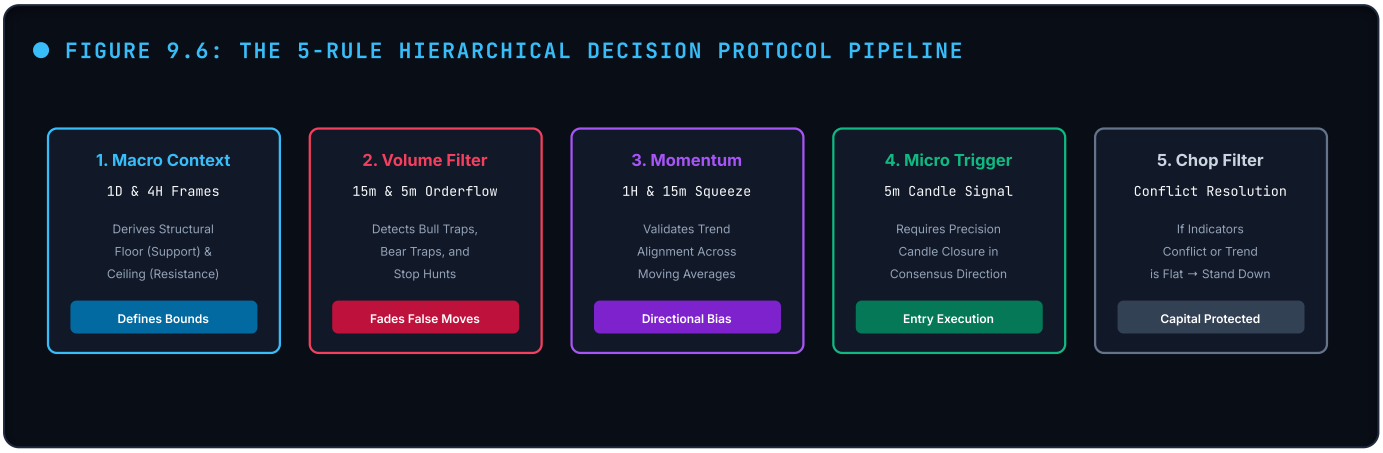
All generated algorithms must conform to Cryptonite's abstract syntax tree (AST) safety enforcer:

1. Must import and inherit from `CryptoniteStrategyConfig` and `CryptoniteStrategy`.
2. **STRICT AST COMPLIANCE:** Direct calls to `self.submit_order()`, `self.cancel_order()`, or direct access to `self.portfolio`, `self.msgbus`, or `self.exec_engine` are strictly prohibited.
3. All order submissions must use safe, risk-managed helpers: `self.buy_market()`, `self.sell_market()`, `self.close_all()`, `self.is_flat()`, `self.is_long()`, `self.is_short()`.

> **1-Click Registration Workflow**
>
> When the agent outputs executable strategy code, the UI chat panel renders an integrated green **Save Strategy** button. Clicking this instantly registers the strategy in PostgreSQL, making it immediately selectable in the **Research Lab** for backtesting!

9.5 The 5-Rule Hierarchical Execution Protocol (StrategistEngine)

To eliminate emotional bias and enforce institutional capital preservation, the Strategist Agent's deterministic engine (`StrategistEngine`) analyzes multi-timeframe snapshots using a strict 5-layer hierarchy:



Rule Protocol Specifications

Layer	Rule / Protocol	Timeframe	Condition & Mathematical Logic	Execution Outcome
1. Macro Context	Floor & Ceiling	`1D` & `4H`	Derives structural floor (min(Low_1D, Low_4H)) and ceiling (max(High_1D, High_4H)) relative to mark price.	Rejects long entries into overhead ceilings or shorts into support floors.
2. Volume Trap	Trap / Hunt Filter	`15m` & `5m`	Detects bull traps (price > upper BB on declining volume) and stop hunts (massive volume spike with long wicks).	**Fades the breakout:** generates counter-trend bias or aborts trade.
3. Momentum Layer	Trend Alignment	`1H` & `15m`	Requires EMA alignment (EMA_20 > EMA_50) on both 1H and 15m intervals.	Sets directional bias: `LONG` or `SHORT`.
4. Micro Trigger	Entry Confirmation	`5m`	Validates precision 5m candle close conforming to momentum bias.	Calculates exact entry price, stop-loss, and take-profit target.
5. Chop Avoidance	Capital Protection	All Frames	Triggered whenever indicators conflict or volatility drops below threshold.	Outputs **CHOP** (flat/no trade), preserving capital.

10. Dynamic System Settings & AI Fleet Configuration

The **Settings Dashboard** enables runtime customization of AI agent prompts, model parameters, and environment health auditing without restarting microservice containers.

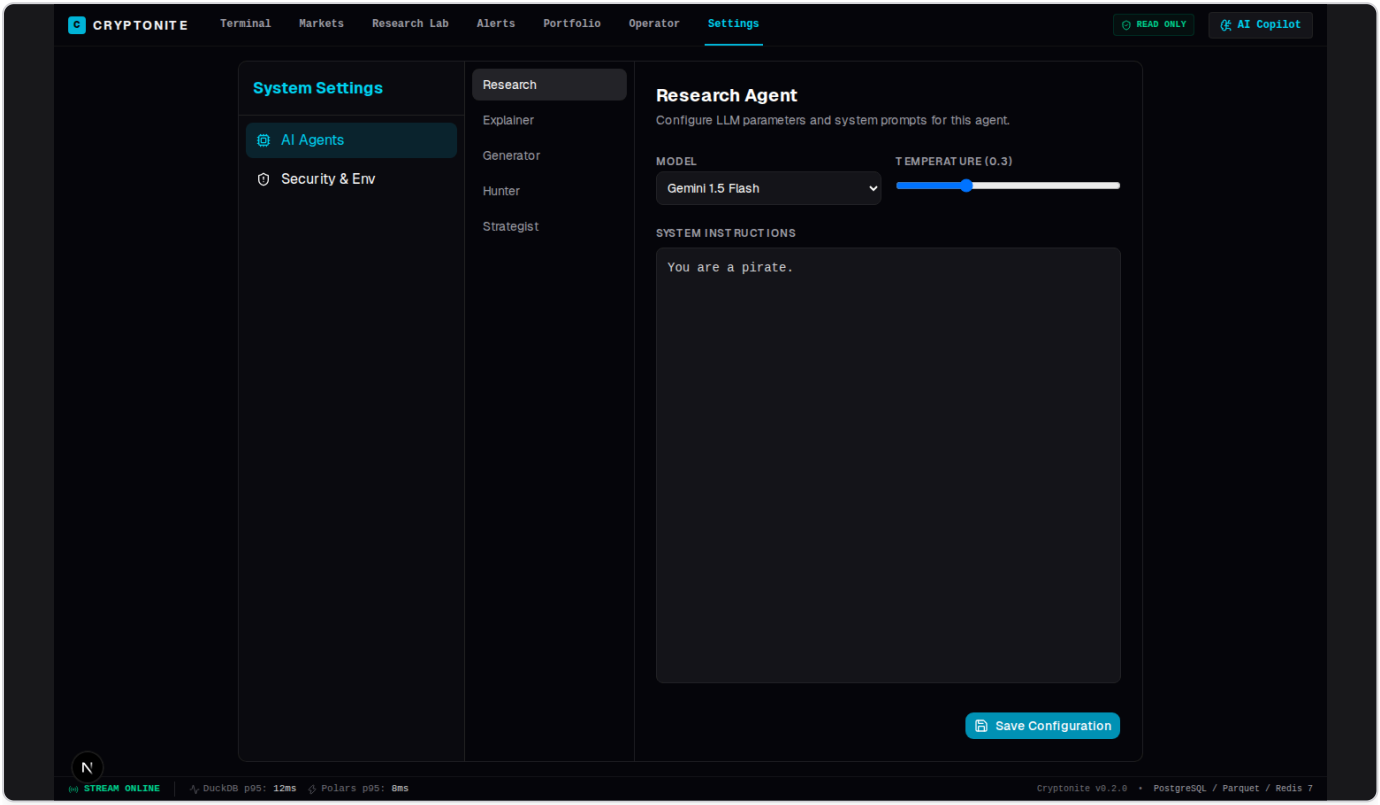


Figure 10.1: AI Agent Settings Dashboard with Model Selection (Gemini 2.5 Pro vs Flash) and System Instructions.

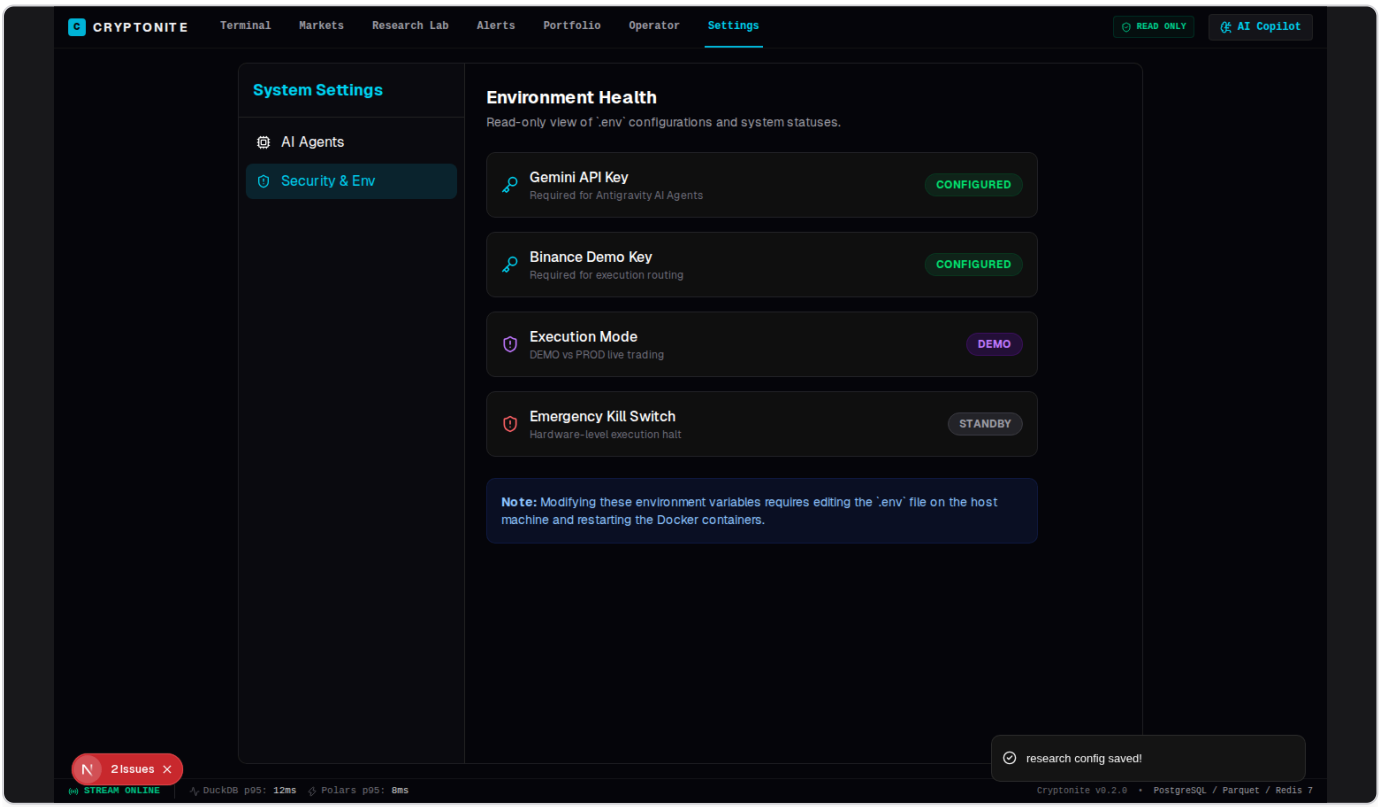


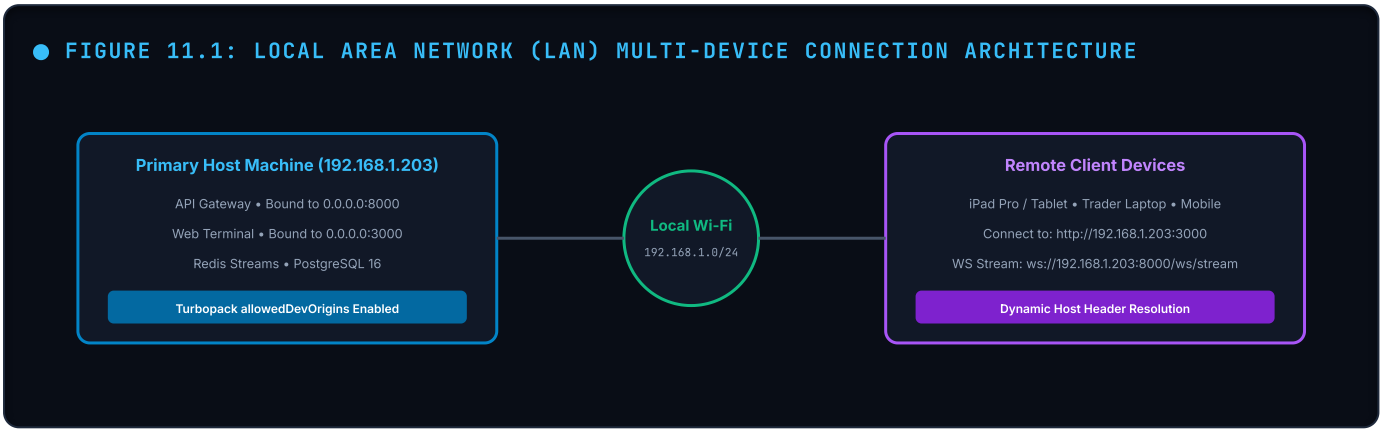
Figure 10.2: Environment Health Audit and Toast Notification Confirming Saved Agent Configuration.

Configuration Capabilities

- **Per-Agent Customization:** Select between `Research`, `Explainer`, `Strategist`, `Generator`, and `Hunter` tabs to tune individual personas.
- **Model Choice Dropdown:** Toggle between `Gemini 2.5 Flash` (optimized for sub-second, low-latency queries) and `Gemini 2.5 Pro` (optimized for complex, multi-step quantitative reasoning and AST validation).
- **Temperature Slider:** Adjust model randomness between `0.0` (strictly deterministic and analytical) and `1.0` (creative and exploratory).
- **System Instructions Editor:** Directly edit agent prompts in the text area. Clicking **Save Configuration** persists changes to the PostgreSQL `agent_configs` table, which refreshes within 60 seconds across all worker pods.
- **Security & Env Health Audit:** Inspects active environment variables (`GEMINI_API_KEY`, `BINANCE_DEMO_KEY`, `Execution Mode`, `Kill Switch Status`) with visual badges (`CONFIGURED` / `STANDBY`).

11. Local Network (LAN) & Multi-Device Access Architecture

Cryptonite is engineered for seamless operation from secondary devices (laptops, iPads, smartphones) on your local network (LAN):



Configuration Summary

Interface	Host Binding Address	Port	Remote Device Access URL
Web User Interface	`0.0.0.0` (All interfaces)	`3000`	`http://192.168.1.203:3000`
API & WebSocket Gateway	`0.0.0.0` (All interfaces)	`8000`	`ws://192.168.1.203:8000/ws/stream`

Steps for Remote LAN Access

1. Ensure your secondary device is connected to the same local Wi-Fi subnet (`192.168.1.0/24`).
2. Open any modern browser (Chrome, Safari, Firefox) on your device.
3. Navigate to: `http://192.168.1.203:3000`.
4. Log in with operator credentials: `admin` / `admin123` / TOTP: `000000`.
5. Live WebSocket price streams and AI multi-agent chats route automatically over the LAN IP.

12. Operational Troubleshooting & Diagnostic Runbooks

When diagnosing system issues, adhere to the following verified diagnostic procedures:

Diagnostic Runbooks

Symptom	Probable Root Cause	Diagnostic Command	Corrective Action
AI Copilot shows "API Key Required"	`GEMINI_API_KEY` missing or empty in root `.env` file.	`docker compose exec agent_worker env | grep GEMINI`	Add valid key to `.env` and run `docker compose restart agent_worker`.
Backtester reports "Out of order bars"	Historical parquet files contain non-monotonic timestamps.	`uv run pytest tests/test_full_stack_backtest_e2e.py`	The updated `cadence.py` engine automatically deduplicates and sorts timestamps.
WebSocket disconnects on LAN device	Firewall blocking port 8000 or client resolving localhost.	`curl http://192.168.1.203:8000/health`	Allow incoming connections on port 8000; verify `ws.ts` hostname resolution.
Kill-Switch tripped (HTTP 423 Locked)	Circuit breaker tripped due to drawdown or operator manual trip.	`curl -X GET http://localhost:8000/api/v1/control/kill-switch/status`	Authenticate as operator and submit step-up password to reset latch generation.

> **Complete System Health Check Command**
> To verify all containers are operational, run:
> `cd infra && docker compose ps`
> All 8 microservices (`postgres` , `redis` , `api` , `ingest` , `scanner` , `backtester` , `executor` , `agent_worker`) should show status `Up` or `Healthy` .